==========

==========

CHAPTER 11 — VIRTUALIZATION PIPELINE (STAGE-1 + STAGE-2 INTEGRATION)

==========

==========

A maximum-detail, RTOS/QNX-style technical breakdown of how ARMv8 integrates Stage-1 and Stage-2 translation, merges permissions and attributes, handles TLB behavior, and enforces VM isolation.

****

****

11.1 — Introduction to the Virtualization Pipeline

****

****

ARMv8 virtualization relies on a **two-stage translation pipeline**:

```
Stage-1 (Guest OS): VA → IPA
Stage-2 (Hypervisor): IPA → PA
```

The pipeline ensures:

- VM isolation
- guest OS independence
- hypervisor control over real memory
- safe device access
- merged permissions
- merged memory attributes
- deterministic behavior

Stage-1 is controlled by EL1 (guest OS). Stage-2 is controlled by EL2 (hypervisor).

****

****

11.2 — Why Two Stages Are Necessary

****

****

Two-stage translation enables:

Virtualization

Guest OS sees a “physical” memory (IPA), but hypervisor maps IPA → real PA.

Security

Hypervisor can:

- deny access
- force non-exec
- force read-only
- force Device memory
- isolate VMs

Flexibility

Guest OS can manage its own virtual memory without knowing real hardware layout.

Resource Management

Hypervisor can:

- overcommit memory
- balloon memory
- swap pages
- migrate VMs
- remap IPA → PA dynamically

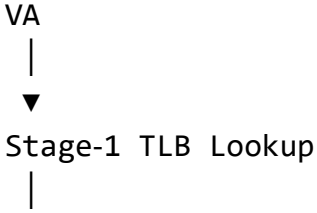
****

****

11.3 — Full Translation Pipeline Overview

****

****



```
 ├── Hit → IPA
 ▼
Stage-1 Walk → IPA
 │
 ▼
Stage-2 TLB Lookup
 │
 ├── Hit → PA
 ▼
Stage-2 Walk → PA
 │
 ▼
Merge Permissions + Attributes
 │
 ▼
Insert into TLB
 │
 ▼
CPU Access
```

****

****

11.4 — Stage-1 Output: IPA

****

****

Stage-1 produces:

- **IPA_base** (from descriptor)
- **offset** (from VA)
- **permissions** (AP)

- **execute permissions** (PXN/UXN)
- **memory attributes** (AttrIndx → MAIR_EL1)
- **shareability** (SH)

IPA is not the real physical address.

****

****

11.5 — Stage-2 Input: IPA

****

****

Stage-2 receives:

- IPA
- Stage-1 permissions
- Stage-1 attributes
- Stage-1 shareability

Stage-2 then applies:

- S2AP
- S2PXN/S2UXN
- S2AttrIndx → MAIR_EL2
- S2SH

Teaching Note

Stage-2 overrides Stage-1.

****

****

11.6 — Stage-2 Output: PA

****

****

Stage-2 produces:

- **PA_base** (from descriptor)
- **offset** (from IPA)
- **final permissions**
- **final attributes**
- **final shareability**

PA is the real physical address used by hardware.

****

****

11.7 — Permission Merging (Stage-1 + Stage-2)

****

****

Final permissions = intersection of Stage-1 and Stage-2.

Example 1 — Stage-1 allows, Stage-2 denies

- Stage-1 AP = read/write
- Stage-2 S2AP = read-only

Final = read-only

Example 2 — Stage-1 denies, Stage-2 allows

- Stage-1 AP = no access
- Stage-2 S2AP = read/write

Final = no access

Example 3 — Execute permissions

- Stage-1 UXN = 0 (EL0 executable)
- Stage-2 S2UXN = 1 (EL0 non-exec)

Final = non-exec

Teaching Note

Hypervisor always wins.

****

****

11.8 — Attribute Merging (Stage-1 + Stage-2)

****

****

Final memory attributes = Stage-2 overrides Stage-1.

Example 1 — Guest marks MMIO incorrectly

- Stage-1 AttrIndx = Normal WB
- Stage-2 AttrIndx = Device-nGnRnE

Final = Device-nGnRnE

Example 2 — Guest marks memory non-cacheable

- Stage-1 AttrIndx = Normal NC
- Stage-2 AttrIndx = Normal WB

Final = Normal WB

Example 3 — Guest marks memory as Device

- Stage-1 AttrIndx = Device
- Stage-2 AttrIndx = Normal WB

Final = Device (Hypervisor may override, but device memory must remain device.)

****

****

11.9 — Shareability Merging

****

****

Final shareability = Stage-2 overrides Stage-1.

Example

- Stage-1 SH = Non-shareable
- Stage-2 SH = Inner Shareable

Final = Inner Shareable

Teaching Note

Hypervisor ensures SMP correctness.

****

****

11.10 — TLB Behavior in Virtualization

****

****

TLB entries store:

- VA → IPA → PA
- merged permissions
- merged attributes
- ASID
- VMID

TLB lookup requires:

```
VA + ASID + VMID
```

Implementation Note

TLB entries contain final merged results.

****

****

11.11 — Walk Cache Behavior in Virtualization

****

****

Walk cache stores:

- Stage-1 intermediate descriptors
- Stage-2 intermediate descriptors

Walk cache must be invalidated when:

- Stage-1 tables change
- Stage-2 tables change
- hypervisor remaps IPA → PA
- BBM sequences occur

Teaching Note

TLBI invalidates walk cache.

****

****

11.12 — Virtualization Fault Types

****

****

Faults include:

Stage-1 Faults

- Translation Fault
- Access Flag Fault
- Permission Fault
- Execute Fault
- Alignment Fault

Stage-2 Faults

- S2 Translation Fault
- S2 Access Flag Fault
- S2 Permission Fault
- S2 Execute Fault
- S2 Alignment Fault

Delivery

Stage-1 faults → guest OS Stage-2 faults → hypervisor

****

****

11.13 — Virtualization Example (Full Walkthrough)

****

****

Given:

`VA = 0x0000_7FFF_1234_5678`

Stage-1

- TTBR0_EL1 selected
- L1 → table
- L2 → block
- IPA_base = 0x0000_2000_0000_0000
- AP = read/write
- SH = Inner Shareable
- AttrIndx = Normal WB

IPA = IPA_base + offset

Stage-2

- TTBR0_EL2 selected
- L1 → table
- L2 → block
- PA_base = 0x4000_0000_0000_0000

- S2AP = read-only
- S2SH = Inner Shareable
- S2AttrIndx = Device-nGnRnE

PA = PA_base + offset

Final Permissions

read-only

Final Attributes

Device-nGnRnE

****

****

11.14 — Virtualization and Device Memory

****

****

Hypervisor maps:

- MMIO
- PCIe BARs
- interrupt controllers
- DMA buffers
- firmware regions

Device memory must be:

- Device-nGnRnE
- non-cacheable

- strongly ordered

Hypervisor enforces correct attributes.

****

****

11.15 — Virtualization and Memory Overcommit

****

****

Hypervisor may:

- map same PA to multiple IPAs
- remap IPA to different PA
- swap pages to disk
- compress pages
- balloon memory

Stage-2 enables advanced memory management.

****

****

11.16 — Virtualization and Live Migration

****

****

Hypervisor uses Stage-2 to:

- track dirty pages
- remap pages
- freeze guest memory
- copy memory to destination host

AF and S2AP are used for dirty tracking.

****

****

11.17 — Virtualization and Security

****

****

Hypervisor enforces:

- VM isolation
- non-exec pages

- read-only pages
- device memory protection
- kernel integrity
- guest sandboxing

Stage-2 is the primary security boundary.

****

****

11.18 — Virtualization and QNX/RTOS Behavior

****

****

QNX uses Stage-2 for:

- partitioned VMs
- safety-critical isolation
- deterministic memory mapping
- device partitioning

QNX may use static VMID ranges.

****

****

11.19 — Chapter Summary

****

****

This chapter covered:

- full virtualization pipeline
- Stage-1 → Stage-2 integration
- permission merging
- attribute merging
- shareability merging
- TLB behavior
- walk cache behavior
- fault types
- full examples
- device memory
- memory overcommit
- live migration
- security
- QNX/RTOS considerations